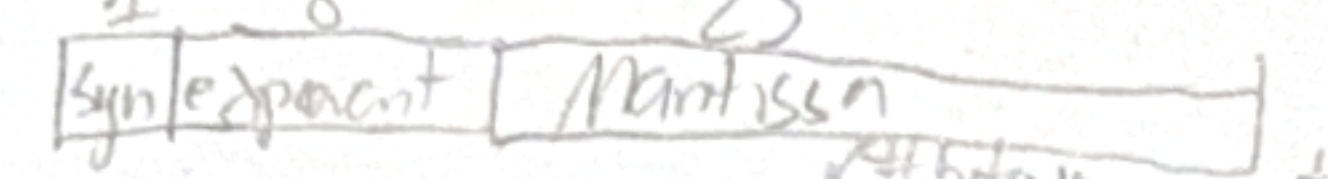


# Number Rep.

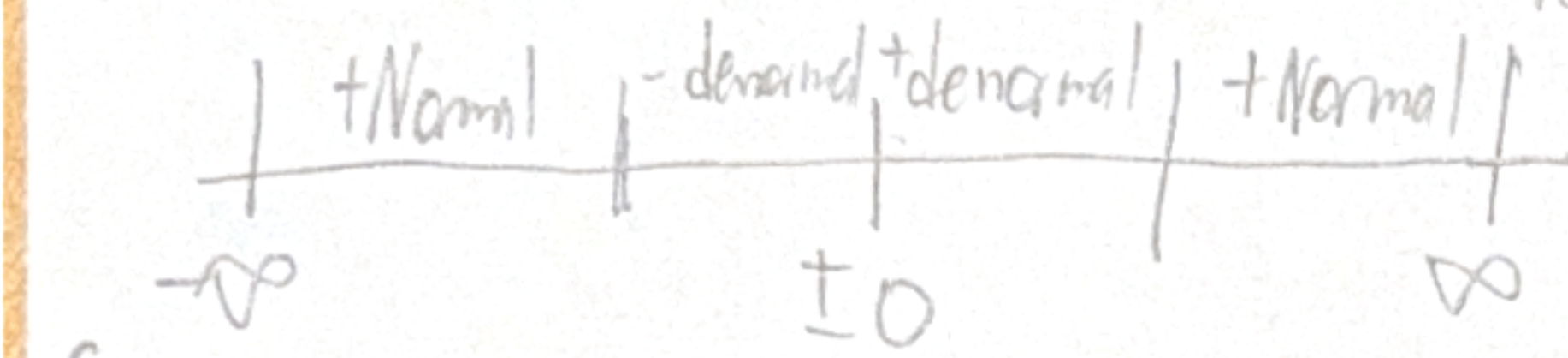
Unsigned:  $[0, 2^n - 1]$ , no negative numbers, arithmetic done mod  $2^n$  to account for overflow  
 Sign-magnitude: 1st bit stores sign, (1 for negative), the rest stores the value. 2 diff. reps of 0, arithmetic requires special handling for the sign bit.  
 Bias: Unsigned number minus some predefined bias, commonly  $(2^{n-1} - 1)$ , comparison easy, arithmetic hard  
 Two's complement: first digit tells sign. Negative numbers are  $-2^{n-1}$ . Negation: flip the bits add 1. Arithmetic easy.  
 Range  $[-2^{n-1}, 2^{n-1} - 1]$

## Floating Point

IEEE 754:   
 - Exponent is biased by  $(2^{n-1} - 1)$   
 Normal:  $\pm 1 \cdot \text{Mantissa} \times 2^{\text{exponent} + \text{bias}}$   
 Denormal:  $\pm 0 \cdot \text{Mantissa} \times 2^{\text{bias} + 1}$

| Exponent | Mantissa/Significand | Meaning  |
|----------|----------------------|----------|
| 0        | Anything             | Denormal |
| 1-254    | Anything             | Normal   |
| 255      | 0                    | Infinity |
| 255      | Nonzero              | NaN      |

- Floating point numbers maintain the order of sign-mag numbers



Struct ex {  
 int a;  
 int b;  
 }  
 struct ex obj;  
 typedef struct {  
 int a;  
 int b;  
 } ex;  
 ex obj;  
 Union student {  
 int age;  
 char [40] name;  
 double gpa;  
 }

C  
 int \*p; # p is a pointer to an int  
 p = &y; # assign the address of y to p  
 z = \*p # assign the value at the address in p to z.

- An array is just a pointer to the first element  
 Pointer arithmetic: pointer + n actually adds  $n * \text{sizeof}(\text{what the pointer is pointing to})$   
 Strings are arrays of chars, the null terminator '\0' signifies the end

Memory Management Segfault: if you try to read or write to an illegal memory location  
 malloc(size\_t size): allocates block of size bytes, returns ptr  
 calloc(size\_t count, size\_t size): allocates block of count \* size bytes, clears contents, returns ptr  
 realloc(void \*ptr, size\_t size): resizes a previously-allocated block, returns the start of the resized block  
 free(void \*ptr): deallocates a block of memory which starts at ptr

Code: the bytecode to be executed, includes constants that are "built-in" to the code:  $(x = y + 1)$  in code  
 Static/data: string literals, global variables. Never changes over program lifetime.  
 Stack: Temporary storage, after a function executes all data in that stack frame is lost. Contains local variables.  
 Heap: for variables that need to persist across functions (grows up)

## Endianness

Big endian: write the MSB in the smallest address  
 Little endian: write the MSB in the largest address (default)  
 NOTE: string is always stored in little endian order.

char \*str1 = "Hello World" ← Hard coded, goes in static  
 char str2[] = "Hello World" ← local var, goes in stack  
 char \*str3 = malloc(12); ← heap  
 strcpy(str3, "Hello World")

RISC-V: where the ram address will be stored  
 jal rd, Label ← the label that we want to jump to  
 jalr rd, rs, imm ← rs = source register, imm = offset to next instruction  
 Pseudoinstruction: ret = jr ra = jalr ra, imm(rs)  
 Useful trick for negating to 0:  
 xori t0, t0, -1  
 addi t0, t0, 1  
 16 (load byte), sign extend across register.  
 XXXXXXX XXXXXXX XXXXXXX XXXXXXX XXXXXXX XXXXXXX XXXXXXX XXXXXXX



Shift right, fill empty spaces with 0s. Not the same as dividing by 2, fails for odd negative numbers.  
Calling Convention  
 sp, bp, s0-s11, ra preserved across function call. If we want to use s0-s12, we should save it on the stack.  
 a0-a7, ra, t0-t6 not preserved across function call.

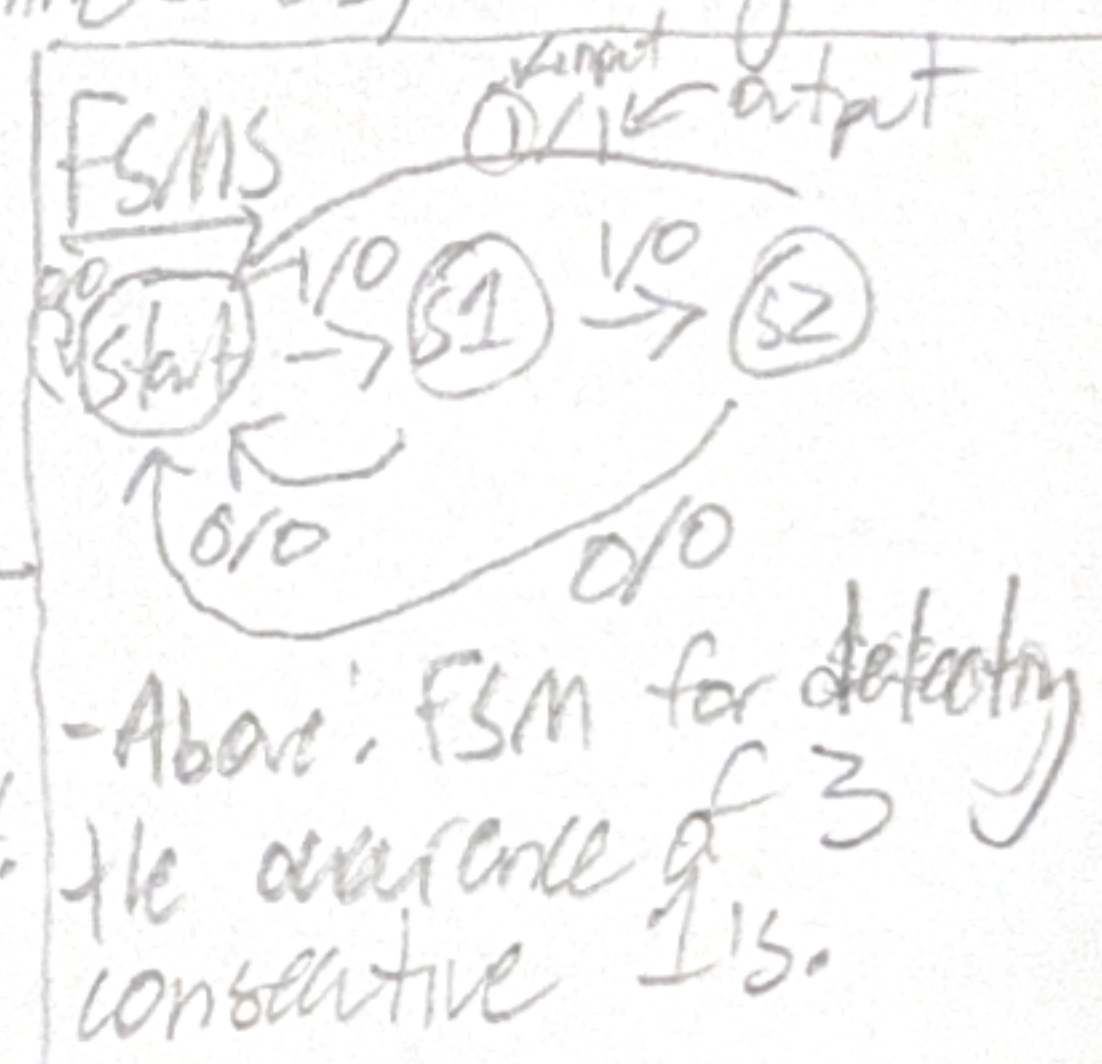
Ex: #Prologue #Epilogue  
 addi sp sp -12 lw s0 0(sp)  
 sw s0 0(sp) lw s1 4(sp)  
 sw s1 4(sp) lw ra 8(sp)  
 sw ra 8(sp) addi sp sp 12

Helpful Conversions

|            |                 |                    |
|------------|-----------------|--------------------|
| $2^0 = 1$  | $2^7 = 128$     | $0 = 0b0000 = 0x0$ |
| $2^1 = 2$  | $2^8 = 256$     | $1 = 0b0001 = 0x1$ |
| $2^2 = 4$  | $2^9 = 512$     | $2 = 0b0010 = 0x2$ |
| $2^3 = 8$  | $2^{10} = 1024$ | $3 = 0b0011 = 0x3$ |
| $2^4 = 16$ | $2^{11} = 2048$ | $4 = 0b0100 = 0x4$ |
| $2^5 = 32$ | $2^{12} = 4096$ | $5 = 0b0101 = 0x5$ |
| $2^6 = 64$ | $2^{13} = 8192$ | $6 = 0b0110 = 0x6$ |
|            |                 | $7 = 0b0111 = 0x7$ |
|            |                 | $8 = 0b1000 = 0x8$ |

CALL  
 Compiler: converts C to RISC-V. Optimizations occur at this level. Output will contain labels, pseudoinstructions, and relative addressing.  
 Assembler: RISC-V  $\rightarrow$  object file. Takes 2 passes. One to remember the positions of the labels and the other to use label positions to generate machine code. Generates 2 tables:  
 Symbol table: labels and their relative addresses (where they are defined)  
 relocation table: parts of code that will need to be calculated & changed later. (external labels, static data, etc)  
 $\rightarrow$  Outputs: .o file. Includes an object file, header (metadata), text segment (machine code), data segment (static data), relocation and symbol tables, and debugging information.  
 Linker: takes in multiple .o files  $\rightarrow$  .exe file (executable). Resolves all references.  
 Loader: .exe file  $\rightarrow$  puts the program on memory and gets it ready to run.

Combinational Logic  
 AND  $\Rightarrow$  D- OR  $\Rightarrow$  D- NOT  $\Rightarrow$  D- XOR  $\Rightarrow$  D- NAND  $\Rightarrow$  D- NOR  $\Rightarrow$  D-  
 AND, OR, and NOR can be cascaded (e.g.  $A \cdot B \cdot C = (A \cdot (B \cdot C))$ ). NAND cannot.

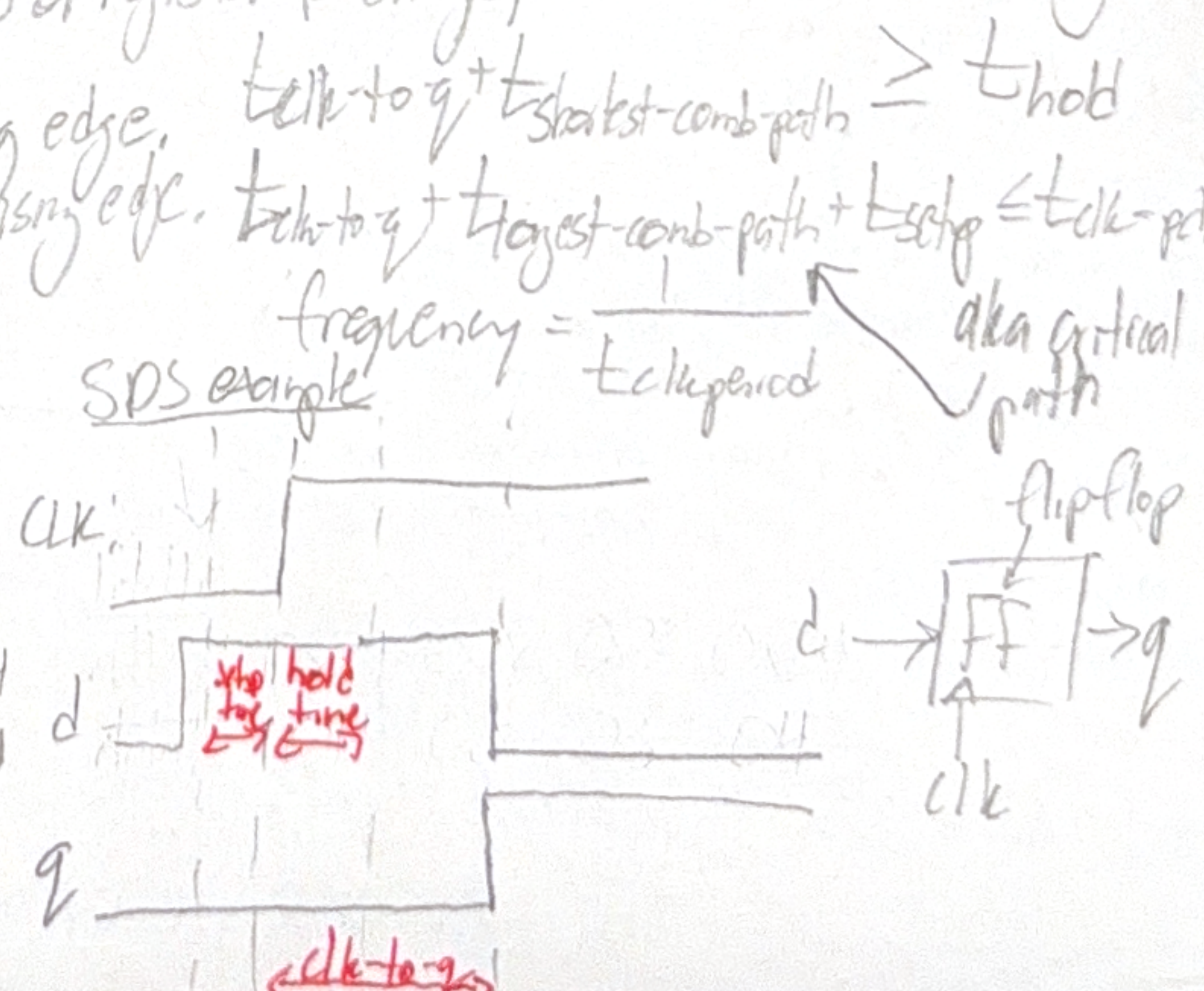


SPS  
 CLK - steady square wave that synchronizes the system  
 setup time - When the input must be stable before the rising edge of the clock  
 hold time - When the input must be stable after the rising edge of the clock  
 CLK-to-q - how long it takes the output of a flip flop or register to change, measured from the rising edge of the clock  
 flip flop - one bit of state that samples every rising edge.  
 register - several bits of state that sample on a rising edge.

More Helpful Conversions      Endianness Ex

|                     |              |
|---------------------|--------------|
| $9 = 0b1001 = 0x9$  | $0x01020304$ |
| $10 = 0b1010 = 0xA$ |              |
| $11 = 0b1011 = 0xB$ |              |
| $12 = 0b1100 = 0xC$ |              |
| $13 = 0b1101 = 0xD$ |              |
| $14 = 0b1110 = 0xE$ |              |
| $15 = 0b1111 = 0xF$ |              |

big endian: 01 02 03 04  
 little endian: 04 03 02 01





manager node (Caches)  
 int nextTask = 0; MPI\_Status status; MPI\_Send(&message, 1, MPI\_INT32\_T, MPI\_ANY\_SOURCE, 0, MPI\_COMM\_WORLD);  
 while (nextTask < numTasks) { assign tasks; MPI\_Recv(&message, 1, MPI\_INT32\_T, MPI\_ANY\_SOURCE, 0, MPI\_COMM\_WORLD, &status); message = nextTask; MPI\_Send(&message, 1, MPI\_INT32\_T, MPI\_ANY\_SOURCE, 0, MPI\_COMM\_WORLD); nextTask++; }  
 MPI\_Recv(&message, 1, MPI\_INT32\_T, MPI\_ANY\_SOURCE, 0, MPI\_COMM\_WORLD, &status);  
 message = TERMINATE; MPI\_Send(&message, 1, MPI\_INT32\_T, MPI\_ANY\_SOURCE, 0, MPI\_COMM\_WORLD);

Temporal locality - if a memory location is referenced then it will tend to be referenced again soon  
 Spatial locality - if a memory location is accessed, then nearby addresses will tend to be referenced soon  
 Fully-associative cache - [tag | offset]  
 One comparator per block, each mem. block can go anywhere.  
 N-way set-associative - N slots assigned to each index  
 N. # sets = # blocks  
 A given memory block can go in N places  
 Direct-mapped - each mem. block can only go in 1 place.  
 cache size = block size \* num blocks  
 # offset bits = log2 (block size)  
 # index bits = log2 (# indices)  
 # tag bits = # bits in mem. address - # index bits - # offset bits  
 Eviction policies - LRU, MRU, FIFO, LIFO, random  
 Cache write policies - write-back, only write entry to main memory when it's evicted from the cache. (faster, less resistant to failure)  
 Write-through, write entry to memory every time it's changed. (slower, but no inconsistencies).

Types of cache misses  
 1. Compulsory: a miss that must occur when you bring in a block for the first time  
 2. Conflict: if the block was fetched before, but was evicted before the cache was full  
 3. Capacity: occurs if the block was fetched before, but evicted while the cache was full  
 AMAT = hit time + (miss rate \* miss penalty)  
 LZ AMAT: L1 hit time + (L1 miss rate \* (L2 hit time + mem access time \* L2 miss rate))  
 OS jobs  
 - start services  
 - provide interaction / outside world  
 - loads, runs & manages programs  
 - running multiple processes is achieved by context switching.

I/O  
 - control register, says its OK to read/write. Device register: contains data  
 Polling: repeatedly read control register in a loop. Good for always active I/O devices w/ low latency.  
 Interrupt: I/O device pings CPU when data is ready. CPU can initiate data transfer + have DMA take care of the rest.  
 VM - gives each program the illusion of full address space. (not any program has access to the whole physical memory).  
 # bits in offset = log2 (size of page)  
 # bits in PPN = log2 (size of PM) - # offset bits  
 # bits in VPN = log2 (size of VM) - offset bits  
 # bits in PTE = # PPN bits + # metadata bits

VPN-PPN translator:  
 1. Program executes a load specifying a VA  
 2. Computer translates VA to the physical memory address, the page table  
 3. If mapping ONE throws a page fault allocate a new page  
 4. First, look in TLB, then look in pg. table then disk (slowly in disk)  
 5. OS reads memory at the PA and returns the data to the program  
 TLB: TLB Valid | VPN | Dirty | Access bits | PPN  
 PTE: (PPN bits + metadata)

Dependability MTBF = MTTF + MTTR. Availability = MTTF / (MTTF + MTTR)  
 Error correction - Hamming distance: difference in # of bits.  
 Hamming ECC: interleaved parity bits w/ data. M parity bits cover 2^M - M - 1 data bits. The incorrect bit.  
 For n-bit error correction, we need each valid codeword to have a Hamming distance of 2n+1. for detection, need n+1.  
 RAID - Redundant Arrays of Inexpensive Disks  
 0 - No redundancy, stripe data across multiple disks  
 1 - mirror the disks  
 3 - parity disk block-level striping (one disk devoted entirely to parity) - fast small reads  
 5 - block-level striping, but distribute across all disks (fast small writes)  
 6 - have another parity block, and also distribute parity across all disks. Robust to 2 disk failures.

WCL: power usage effectiveness = total building power / IT equipment power  
 MapReduce map: (in-key, in-value) -> (interm-key, interm-value)  
 reduce: (interm-key, list(interm-value)) -> list(out-value)  
 e.g. ("I do I learn") -> ("I", 1), ("do", 1), ("I", 1), ("learn", 1)  
 e.g. ("I", [1, 1]) -> ("I", 2)  
 PASC-V alignment: an n-byte field must start at an n-byte aligned boundary



Path state element - element that is connected to the clock

IMEM - read-only memory for fetching instructions

DMEM - memory for loading (read) and storing (write) data words

Control signals ImmSel, ASel, Bsel, Branching, and WB Sel can be \*, others cannot

Stages:

- Instruction fetch: send address to IMEM, read IMEM at that address. Uses PC register + 4 adder, PC select, PC incrementer
- Instruction Decode: generate control signals from instruction bits, generate immediate, read registers from RegFile, uses RegFile, ImmSel
- Execute: perform ALU operations, do branch comparison. Uses ASel + Bsel MUX, branch comparator, ALU
- Memory: read from or write to DMEM, uses DMEM
- Writeback: write back either PC+4, ALU result, or data from memory to the RegFile.

Throughput: # instructions per unit of time. Latency: how long it takes to execute a single instruction.

Pipeline Hazards IF-ID-EX-MEM-WB

- ① Structural - hardware does not support access across multiple instructions in a single cycle. Occurs when multiple instructions compete for access to a single physical resource. Sol'n: stall or add more hardware.
- ② Data - instructions have data dependency (either EX-ID or MEM-ID). Need to wait for previous instruction to complete its read/write. Solutions: stall, forward data to next operations, rearrange instructions.
- ③ Control - flow of execution depends on the previous instruction. If we take a branch, we must flush the pipeline and stall (typically 3 cycles). CPU performance can be improved w/ branch prediction.

Double-pumping - in one cycle, data is written to the RegFile in the first half and read from the RegFile in the second half.

Parallelism

Amdahl's Law:  $speedup = \frac{1}{(1-F) + \frac{F}{S}}$ ; F = fraction of program that is optimized, S = speedup factor of optimized portion

SIMD (single instruction, multiple data): use specialized vector registers which store 128, 256, or 512 bits.

DO NOT FORGET TAIL CASE

MIMD (multiple instruction, multiple data) aka multithreading - each thread has its own registers, PC, stack, but all threads share the heap. #pragma omp parallel 3: each thread should run everything in this block.

#pragma omp critical: only one thread can run this critical segment at a time. (OPEN MPI ex)

#pragma omp barrier: forces all threads to wait until all threads have hit the barrier. #define TERMINATE -1

#define READY 0

SIMD ex (product of all array elements):

```

MIMD ex (assume arr has length 8n)
double fast_product(double* arr, int n) {
    double product = 1;
    #pragma omp parallel for
    for (int i = 0; i < n; i++) {
        double subproduct = arr[i*8] * arr[i*8+1] * ... * arr[i*8+7];
        product *= subproduct;
    }
    return product;
}

```

static int product\_vectorized(int n, int\* a) {

```

    int result[4];
    #pragma omp parallel for
    for (int i = 0; i < n/4*4; i += 4) {
        prod_v = _mm_mulloq_p32(prod_v, _mm_loadu_si128((int*)a + i));
    }
    _mm_storeu_si128((__m128i*) result, prod_v);
    for (int i = n/4*4; i < n; i++) { // handle tail case
        result[0] += a[i];
    }
    return result[0] * result[1] * result[2] * result[3];
}

```

Multiprocessing - processes don't share memory, difficult to communicate, works on as many cores as you have.

Manager: assigns work, informs user of the progress

Worker: receives work from the manager + does it

```

// worker node
int 32-t message;
while (true) {
    // request more work
    message = READY;
    MPI_Send(&message, 1, MPI_INT32_T, 0, 0, MPI_COMM_WORLD);
    MPI_Recv(&message, 1, MPI_INT32_T, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    if (message == TERMINATE) break;
}
return product;

```

Manager node code

```

MPI_Comm_size(MPI_COMM_WORLD, &totalProcs);
MPI_Comm_rank(MPI_COMM_WORLD, &procID);
if (procID == 0) {
    // manager node code
} else {
    // worker node code
}
MPI_Finalize();

```